



Introduction to while loops

If you had to do the same thing over and over and over again... well, you might get a bit loopy. And you'd probably think to yourself, "I wish I could spend my time on something more meaningful." Well, that's where computers can help. They do the looping for you.

As a refresher, a loop is a block of code used to carry out iterations. An iteration is the repeated execution of a set of statements, where one iteration is the single execution of a block of code. And an iterable is an object that's looped, or iterated, over.

Typically, data professionals use for loops and While loops to work with iterables. In this video, we'll focus on While loops. A While loop is a loop that instructs your computer to continuously execute your code based on the value of a condition. Magali and Fido are here to help me explain. Notice that Fido is eating treats while the treat bag is in Magali's lap. Fido stays until Magali puts the treat bag away. Then Fido leaves.

This is a While statement in action. While Magali has the treats, Fido is there. Once the condition of Magali holding the treat bag is no longer met, Fido exits. After all, there are no more treats. While loops work in a similar way to branching IF statements. The difference is that, in While loops, the body of the block can be executed multiple times instead of just once. This is great for avoiding redundancy in code.

Let's review an example. Assign the value of zero to the variable x. As you've learned, we call this action initializing to give an initial value to a variable. Next, start the While loop. Set a condition for this loop that x needs to be less than five. The prior line just initialized it, so this condition is currently true. Then, end the while loop line with a colon. On the next two lines, you may notice a block that's indented to the right. This is the while loop's body. There are two lines in the body of the loop. In the first line, print a message followed by the current value of x. In the second line, increment the value of x by adding one to its current value and assigning it back to x. In the body of the while loop, we told the computer to print the results of each iteration. Notice how each iteration changed the value of x.

Our while loop started at x equals zero, then printed the message... and increments the value of x to one. But because this is a loop, the computer doesn't just continue executing with the next line in the script. Instead, it loops back around to re-evaluate the condition for the While loop. This is the second iteration. And because one here is still less than five, it executes the body of the loop again. The third iteration increments x by one. Now the value of x is two.

The computer will keep doing this until the condition isn't true anymore. Let's add a print statement outside the body of the while loop to let us know the final value of x. When the loop finishes, the next line of code is executed. So, now that x has reached the value five, the loop statement ends. The computer prints the last line of output as x equals five. We can also use the logical operators and, or, and not, that we worked with earlier. Similar to IF statements, using these operators allows us to combine the values of several expressions to get the result we want.

The important thing to remember is that the condition used by the while loop needs to evaluate to true or false. It doesn't matter if this is done by using comparison operators or calling additional functions. I think you're ready for an example that uses several different concepts that you've learned so far, and even a couple that you haven't. To help you explore the example, I'll guide you as we go, and I'll explain the concepts to you. You'll probably find that as you continue on your journey learning Python, you'll encounter new concepts that can be understood in context. It's the same way with spoken language. If I say "the elephant was so gargantuan that it crushed the car by stepping on it," the word "gargantuan" may or may not be in your vocabulary, but the context helps us understand that it has something to do with being "big" or "heavy." It's the same with coding. Pause the video and try to figure out what this code is doing. When you're ready, continue and I'll lead you through it. In this example, we're going to write a short program that generates a random number and then gives the user five chances to guess it. We begin by importing a package called "random." This package has many uses, but the one we want is its ability to generate a random number. Now instantiate a variable called "number," and use the "random int" function from the "random" package to assign it a value.

The value is a random number between 1 and 25, inclusive. Next, we instantiate another variable called "number of guesses" and assign it a value of zero. This variable is important, because it will behave as a counter that controls the logic of our program. Now comes our While loop. We write "While the number of guesses is less than five"... and below it we give the instructions for what should happen at each iteration of the loop. First, the script will print a message telling the user to guess a number from 1 to

25. Then we instantiate a new variable called "guess," which will capture a string of whatever the user inputs.

The input function used here will create a prompt that allows the user to type their guess. The next line converts the guess to an integer. This is important because without it, the input function would capture the number as a string, and it would be impossible to win the game, because even if they guess the right number, the string of the number will never evaluate as being equal to the integer of the number. Now, we increase the "number of guesses" variable by one. This is an important step because, remember, the While loop will continue to iterate for as long as the "number of guesses" variable is less than five. If we left this out, the loop would never stop running and the user would have infinite guesses until they got it right. By the way, this syntax is a helpful shortcut.

Instead of writing "number of guesses equals number of guesses plus one," we can simply write "number of guesses plus-equals one." They mean the same thing, but one way is shorter. Now we're going to have our branching logic that determines what the program does. We want the behavior to be different depending on how many guesses the user has made and whether their guess was correct or not. This code is still within the body of the While loop, so it will execute on every iteration. The first line checks if the guess is correct. If it is, the While loop breaks.

This word "break" is a keyword that lets you escape a loop without triggering any ELSE statement that follows it in the loop. In this case, if the user guesses the correct number, the While loop would break and the code would continue at the IF statement below.

Because the guess is correct, we print a message telling the user that they're correct and include how many tries it took them in the message. But let's go back up to the

While loop and go through what happens if the guess is not correct. We use the ELIF statement to check if the guess was their fifth guess. If it was, we break the While loop. Evaluation would continue below, and because the guess was incorrect, it would trigger the very last ELSE statement that outputs a message telling the user they were unsuccessful and revealing the correct number.

Without this ELIF statement the code would still work... kinda. The user would get a message saying "Nope! Try again," and then immediately get another message telling them they were unsuccessful and revealing the number. But, if the guess was not their fifth guess, the code would proceed to the else statement, which prints "Nope, Try again." Let's run the program and try to guess the number. Congratulations, you just integrated a bunch of concepts into a single script of code and worked on a problem that incorporated some complex logic. These skills are invaluable to data professionals.

Loop

A block of code used to carry out iterations

Iteration

The repeated execution of a set of statements, where one iteration is the single execution of a block of code

Iterable

An object that's looped, or iterated, over

while loop

A loop that instructs your computer to continuously execute your code based on the value of a condition

The condition used by the while loop needs to evaluate to true or false.

break

A keyword that lets you escape a loop without triggering any ELSE statement that follows it in the loop

Question



Fill in the blank: A loop is a block of code used to carry out _____.

- ☒ iterations
- ☐ calculations
- ☐ conversions
- ☐ evaluations

✔ **Correct**

A loop is a block of code used to carry out iterations. Data professionals use loops to automate repetitive tasks.

Continue

Skip